



UNIVERSITY OF
LIVERPOOL

Application Development with C++ (ELEC362)

Lecture 5: Memory management

mihasan@liverpool.ac.uk

Previous lecture

- Cases where the code may not run sequentially were discussed
- If-else and switch-case structures were discussed under selection structures.
- For loop, While and Do-while loops were discussed under repetition structures.
- The concept of event loops was discussed.
- The definition of functions and how they work was discussed.
- Error handling was discussed.

This lecture

- What is covered in this lecture?
 1. Pointers.
 2. Memory managment
- Why is it covered?
 1. Pointers are vital for efficient memory management of any programme.
 2. Memory management is the ultimate point of having pointers.
- How are topics covered in this lecture:
 - 2 source codes

Pointers definition

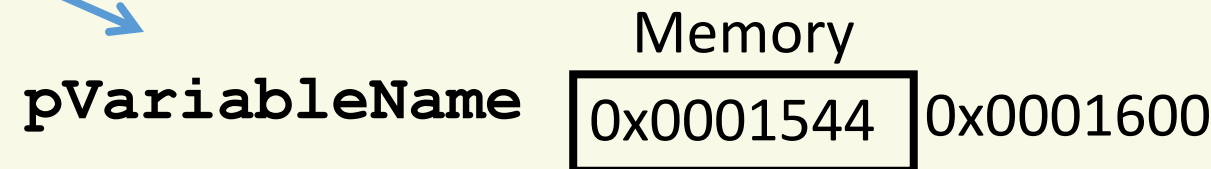
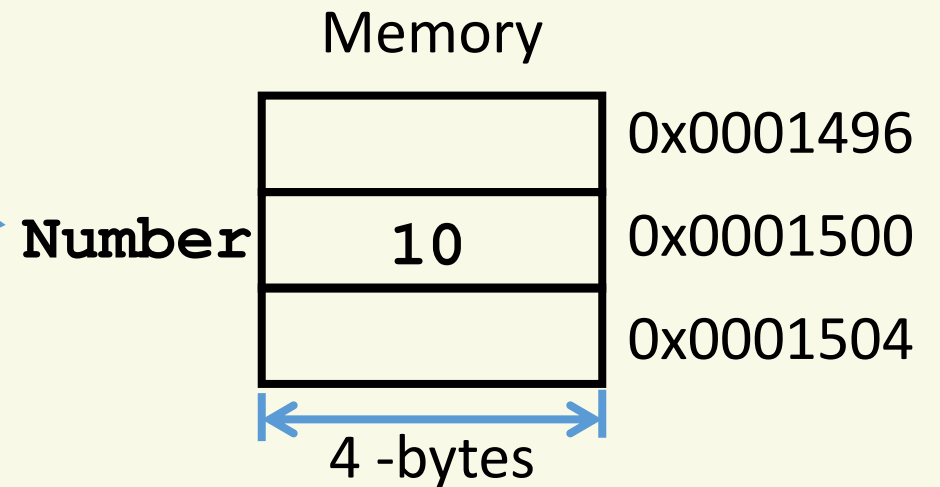
- Pointers are variables that store the memory address of a variable of the same type , or memory location.

```
int Number = 10;
```

- Syntax:

```
DataType *pVariableName;  
DataType* pVariableName;
```

- Google style code requires the dereferencing operator “*” to be attached to either the data type or the variable’s name.



Pointers naming and initialisation

- It is recommended to start the name of a pointer by “p”.
- To initialise a pointer, the referencing operator “&” should be used.
- Examples:

```
int Number{99};  
  
int *pNumber; //Declare pointer  
  
pNumber = &Number; //Initialise it
```

```
int Number{99};  
  
//Declaration & initialization  
int *pNumber{&Number};
```

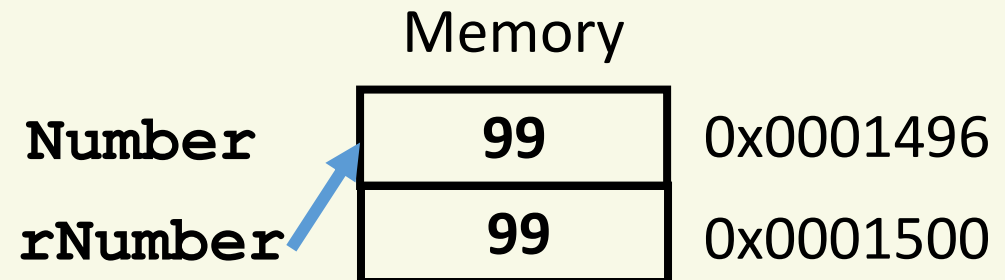
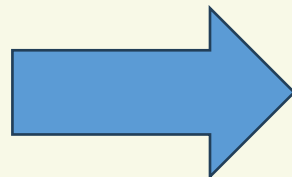
- The datatype of the pointer must match the datatype of the variable it is pointing to.

Referencing and Dereferencing operators

- The two operators to handle pointer are the reference operator “&” and the dereferencing operator “*”, what they do depends on where they appear.

Context	dereferencing operator “*”	reference operator “&”
In a declaration	<code>int *pNumber;</code> Means the variable pNumber must hold a memory address	<code>int &rNumber=x;</code> Means the variable rNumber is another name of the variable x
Elsewhere	<code>cout << *pNumber;</code> Means <u>read the value stored</u> in the memory address held in pNumber.	<code>pNumber=&x;</code> Means <u>read the memory address</u> where the variable x is stored in the memory.

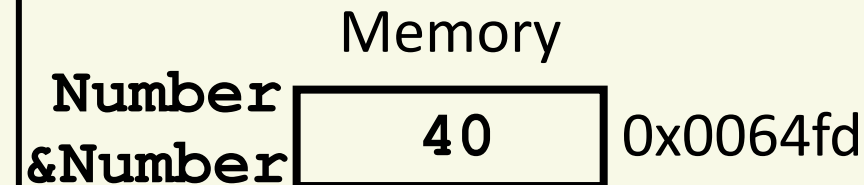
```
int Number=99;  
int &rNumber=Number;
```



Referencing and Dereferencing operators

- Example:

```
int Number = 40;
int *pNumber = &Number; // Say its 0x0064fd
int &rNumber = Number; // "Alias" variable
std::cout << Number << endl;
std::cout << &Number << endl;
std::cout << rNumber << endl;
std::cout << pNumber << endl;
std::cout << *pNumber << endl;
```



- QUESTION: Is it better to use pointers or references?

Pointers and arrays

- The name of any array in C++ is a memory address, which can be used to initialize a pointer to access and manipulate the elements of the array.
- Example:

```
int Data[4]{1,2,3,4};  
int *pData{nullptr}; // Points nowhere  
pData = Data; // Equivalent to pData=&Data[0];  
std::cout<<* (pData+2)<<endl;  
std::cout<<Data<<endl;
```

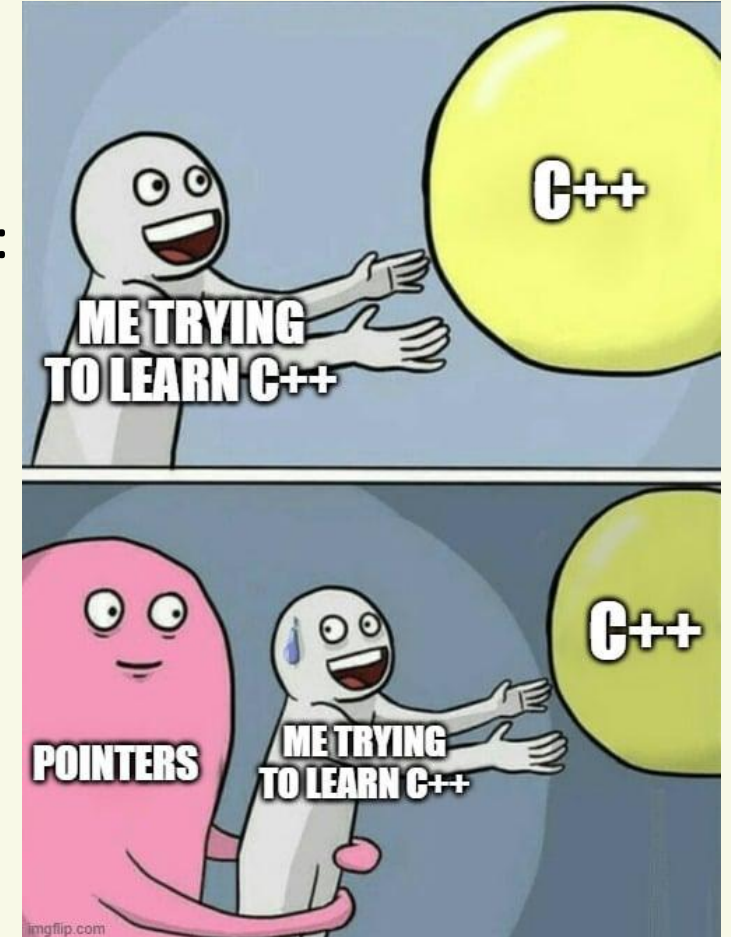
data	
1	0x0001496
2	0x0001500
3	0x0001504
4	0x0001508

Practical note:

- Make sure your code does not access memory locations beyond the array's end.
- After a pointer is not needed, reinitialise it to **nullptr**

Why are pointers a thing?

- Pointers are the key to memory management of a code.
- Mostly this is done by two strategies enabled by pointers:
 1. Dynamic memory allocations: Leaving it to the user to decide how much memory is needed and for how long.
 2. Avoiding copying in memory: using pointer to pass data to functions prevent replication.
- From Lecture 10 onwards, things will be simpler!!



Dynamic memory allocation

- Programs use static memory (the stack) and dynamic memory (the heap).
- Pointers are the only way the programme can have access to dynamic memory.

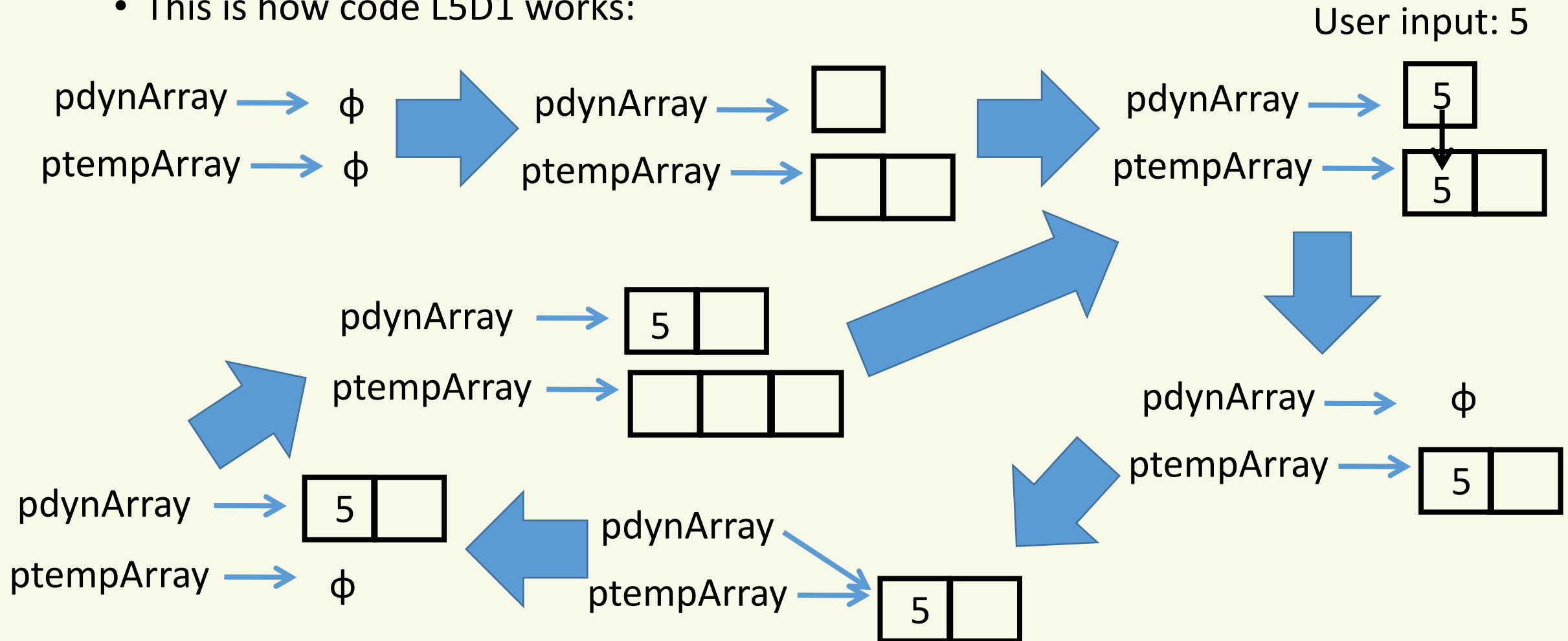
Aspect	Static memory	Dynamic memory
Allocation	Allocation is done at compile-time only	Allocation can be done at compile time or run-time
Lifetime	As long as the programme is running	Can be controlled
Size	Fixed	Variable

- Pointers can access dynamic memory through `new` keyword (for allocation) and `delete` (for freeing memory). `new` and `delete` (`delete []` for arrays) must operate on the same memory location.
- Go to [L5D1.cpp](#)

Practical note: Maximise the use of dynamic memory in your code.

Dynamic memory allocation algorithm

- This is how code L5D1 works:



Passing data to functions using pointers

- In Lecture 4, we discussed simple functions of the type:

```
return_type FunctionName(datatype arg1, datatype arg2) {return output; }
```

- The main limitation of simple functions is that they can only return one variable.
- The use of pointers and references allow us to do this instead:

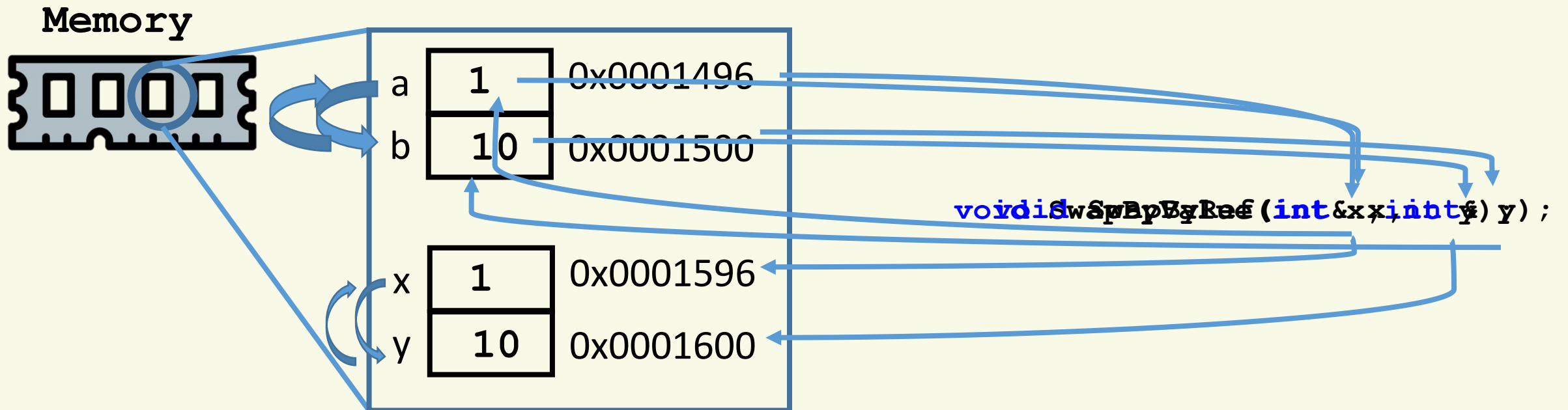
```
void FunctionName(datatype& arg1, datatype& arg2, datatype& arg3, ...) { }
```

- Go to [L5D2.cpp](#)

These can be input and
output at the same
time!!!

Arguments passing in functions

- This is how code L5D2 works:



Practical note: Pass variables by reference or pointers whenever possible.

Professionally written functions

- A lot of libraries written by C++ community use the memory managed style of functions: [GSL Library](#), [SUNDIALS library](#), even [functions in C++ header files](#).
- Sometimes the return of a function is used as an error indicator.

```
int gsl_matrix_fwrite(FILE *stream, const gsl_matrix *m)
```

↑
Return a number
to say how it went,
(0 for success)

↑
Function to write a
GSL matrix to a file

↑
File to be written
to

↑
Matrix to be
written in the file

- To tell the other developers that one of the parameters is an input only, `const` is used.

Function overloading

- Function overloading is defining multiple functions with the same name but different parameters. The compiler can decide which one to call based on the parameters passed at calling.

- Example:

```
// Two integer version
int Max(int x,int y); {return (x > y ? x : y);}
// Three integer version
int Max(int x,int y,int z); {int m = (x > y ? x : y);
                             return (z > m ? z : m);}
//Two double version
double Max(double x,double y); {return (x > y ? x : y);}

void main () {   int c = 2, d = 4;
                 double g = 2.3 , h = 5.2;
                 Max(g,h);
                 Max (c,d);
                 }
```

Function templates

- Function templates are efficient way to generate overloaded functions for different data types.
- The idea is to pass the data type as it is a parameter to the function.

```
template <typename T> // T is the arbitrary datatype
T Max(T x, T y) {return (x > y ? x : y);}

void main () { int c = 2, d = 4;
               double g = 2.3 , h = 5.2;
               char j='y' , k='w' ;
               Max<double>(g,h); // T is double
               Max<int>(c,d);    // T is integer
               Max<char>(j,k);   // T is character
            }
```

Summary

- The definition of pointers, their use and their advantages were discussed.
- Differences between two types of memory were discussed.
- Memory efficient functions were discussed.
- Functions overloading and templates were discussed .